2.6.6 Advanced Topic: Defining Your Own Output Forms

When you introduce a new function into *Mathematica*, one of the things that you can specify is how the function should be printed out.

`Format[`*expr*`] := `*form*	define the output form for *expr*
`Format[`*expr*`, `*type*`] := `*form*	define the output form in a particular format
`FullForm[`*expr*`]`	display the full form of *expr*, ignoring any special output forms

Defining output forms.

This defines `pair` with two arguments to print like a list.

```
In[1]:= Format[pair[x_, y_]] := {x, y}
```

Now `pair` prints as a list.

```
In[2]:= pair[a+b, c]
Out[2]= {a + b, c}
```

The list form was used only for output. Internally, this is still a `pair` function.

```
In[3]:= FullForm[%]
Out[3]//FullForm= pair[Plus[a, b], c]
```

It is important to understand that defining the output form of a function is quite independent of defining the *value* of the function. If you define the value of a function, say by making an assignment like `f[x_]:=`*value*, then every time the function appears in a calculation, *Mathematica* will replace it with the value you defined. On the other hand, defining the output form of a function, say with the assignment `Format[f[x_]]:=`*form*, does not affect the *value* of the function. With this assignment alone, `f[x]` will remain unevaluated when it appears in a calculation. However, when the calculation is finished and *Mathematica* is formatting the output, then any time that `f[x]` appears, it will be formatted in the way you specified.

When you make a definition like `Format[f[x_]]:=`*form*, the value *form* can be any *Mathematica* expression. When `f` is formatted, *form* is evaluated. As a result, you can have *Mathematica* do computations to work out the output form for `f`.

This defines the output form of `rep` to be the result of replicating `x` `n` times.

```
In[4]:= Format[rep[n_]] := Table[x, {n}]
```

The list of four elements is created when `rep[4]` is formatted.

```
In[5]:= rep[4]
Out[5]= {x, x, x, x}
```

rep[4] only *prints* as a list. Internally, it is still just rep[4].

```
In[6]:= 1 + %
Out[6]= 1 + {x, x, x, x}
```

"text"	a string containing arbitrary text
HoldForm[*expr*]	the output form of *expr*, with *expr* maintained unevaluated

Functions that give literal output.

In general, the expression you assign as output forms for functions get evaluated when they are used. Often, however, you want to specify the output form of a function to be some literal object, which should not be evaluated.

One way to do this is to define the output form simply to be a specific string of text. The string can contain special characters, including newlines and so on, as discussed in Section 2.6.9.

Often, however, the arguments of your function have to appear in some way in the output form for the function. You can arrange for this by constructing the expression that represents the output form you want, and then enclosing this expression in a HoldForm function. The HoldForm function prevents the expression inside from being evaluated, but allows its output form to be used.

This defines the function pow to have the printed form of a power, but with the power remaining unevaluated.

```
In[7]:= Format[pow[x_,y_]] := HoldForm[x^y]
```

This prints just like 2^3, but without evaluating 2^3.

```
In[8]:= pow[2, 3]
Out[8]= 2^3
```

You can use assignments like Format[*expr*]:=*form* to specify the output form for any expression or class of expressions. You can, for example, specify output forms for symbols in this way.

This specifies that the output form of the symbol catalan should be the string "G".

```
In[9]:= Format[catalan] := "G"
```

Whenever catalan appears, it is now printed out as G. Notice that in standard *Mathematica* output form, the double quotes around strings are not included.

```
In[10]:= catalan^2/4
                 2
                G
Out[10]= -
                4
```

When you make an assignment for `Format[`*expr*`]`, you are defining the output form for *expr* in the standard *Mathematica* output format (`OutputForm`). By making definitions for `Format[`*expr*`, `*type*`]`, you can specify output forms in other types of output format.

This specifies the `TeXForm` for the symbol `x`.

```
In[11]:= Format[x, TeXForm] := "{\\bf x}"
```

The output form for `x` that you specified is now used whenever the TEX form is needed.

```
In[12]:= TeXForm[1 + x^2]
Out[12]//TeXForm= 1 + {{{\bf x}}^2}
```